

Fine-Tuning ProtBert For Enzyme Classification Tasks

Jason Walker
200571715

Project Supervisor: Katherine James

Abstract

With the growing number of protein sequences being discovered, there is an increasing demand within the field of bioinformatics for novel and efficient methods to classify the function of these proteins. Proteins perform a wide array of functions; such as enzymes which catalyse biochemical reactions. An example enzymatic protein is lactase, which breaks down lactose - a sugar found in milk - into simple sugars. Function classification allows researchers to better understand the significance of newly discovered proteins, enabling discoveries in the fields of health, biology and drug development. This dissertation attempts to explore the capabilities of Protein Language Models for downstream fine-tuning of protein classification tasks, taking into account the computational resources required for training these large models and investigating optimal ways to reduce this computational load whilst retaining prediction accuracy.

This dissertation focuses on the Protein Language Model ProtBERT, a BERT-based (Bidirectional Encoder Representations from Transformers) model trained on a corpus of protein sequence data. ProtBERT was fine-tuned using the Hugging Face Transformers library alongside a preprocessed dataset from the Universal Protein Resource (UniProt) to predict whether a given protein sequence is enzymatic in nature.

Using a Low-Ranking Adaptation (LoRA) approach alongside a publicly available cloud-computing environment (Google Colaboratory) the model yielded strong results; achieving an accuracy rating of upwards of 84.6%, a precision score of 85.3%, recall of 83.8%, and an F1-score of 83.9%. These results demonstrate the potential of leveraging protein language models for protein function classification tasks within bioinformatics.

Declaration

I declare that this dissertation represents my own work, except where otherwise stated.

I also acknowledge the use of AI code-assistance tool GitHub Copilot, integrated within my VS Code local programming environment. Used within the data preprocessing and exploratory data analysis phase to generate code based on typed comments, assisting with manipulating dataframes and generation of visualisations. This was used to discover new ways to manipulate data quickly and to allow me more time to spend on insights. I confirm that any insights gained during this phase of the project are my own.

1. Introduction	6
1.1 Motivation and Rationale	6
1.2 Summary of the Case Study	8
1.3 Aims and Objectives	9
2. Background Review	10
2.1 Transformer Architecture	10
2.2 BERT Architecture	14
2.3 ProtBERT	14
2.4 Tools and Technologies:	15
2.4.1 Data Analysis and Preparation	15
2.4.2 Model Development	15
2.4.3 Model Training	16
2.5. Enzyme Commission (EC) Numbers	16
3. Methodology	17
3.1. Dataset Acquisition	17
3.2. Dataset Analysis and Preparation	18
3.3. Tokenization and Input Encoding	21
3.4. Model Training and LoRA Fine-Tuning Pipeline	22
4. Results	28
4.1. Training Times	28
4.2 Validation Performance During Training	29
4.2.1 Training and Validation Loss Trends	29
4.2.2 Evaluation Metric Trends	30
4.4 Implications Of Results	33
4.5 Considerations For Improved Model Evaluation Data	33
5. Evaluation	34
5.1. Dataset Preparation	34
5.2. Model Selection	34
5.3. Model Fine-Tuning	34
5.4. Model Performance Evaluation	34
5.5. Analysis and Implications	35
6. Conclusions	36
6.1 Personal Reflective Statement:	36
7. References	37
Appendices:	40
Appendix A:	40
Appendix B:	41
Appendix C:	42

1. Introduction

1.1 Motivation and Rationale

Proteins are chains of amino acids which form a protein sequence; this sequence denotes how the protein unfolds in 3-Dimensional space and determines their cellular function. These functions can vary from catalysing metabolic reactions, providing structural support for cells, enabling communication between organisms and more (Morris et al., 2022). Figure 1 shows the distribution of protein categories within the human genome, with most falling into enzymatic and binding proteins.

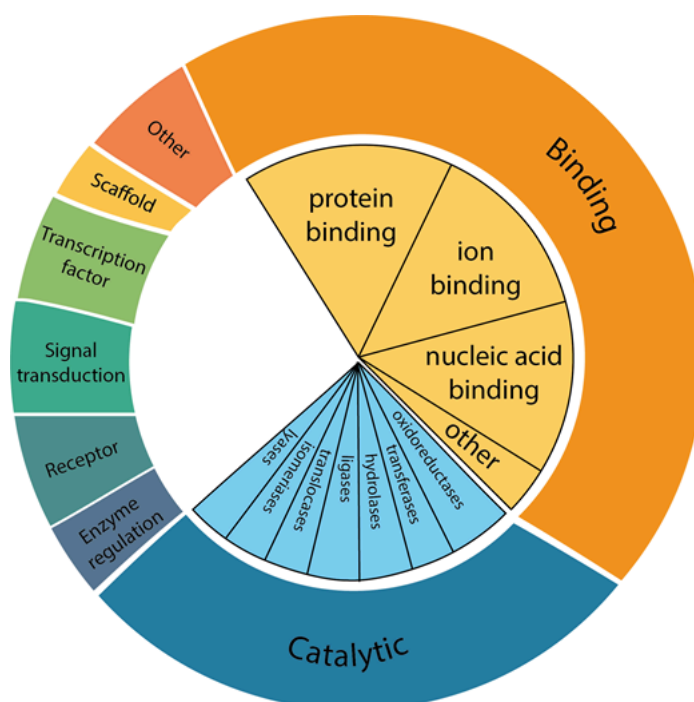


Figure 1.
Classification of human proteins by function
Reproduced from Morris, R., Black, K.A. and Stollar, E.J. (2022).

As proteins are fundamental to all biological processes, a deeper understanding of the functions of proteins is critical for advancing research within the fields of medicine, including drug development, understanding disease mechanisms. The full scope extends beyond these applications and cannot be fully covered in this paper (Zhang et al., 2024).

Within proteomics, new methods are being discovered for the large-scale sequencing of proteins, determining the amino acid chains that comprise them. Traditionally, proteins were sequenced using techniques such as mass spectrometry, where proteases are used to digest proteins into peptides which are then analysed. There are considerable downsides to this approach, due to numerous stages where the experimentation can fail, such as contamination during sample handling (Dupree et al., 2020). Due to this, technologies for single-molecule protein sequences are being constantly innovated, nanopore technologies have proven successful for DNA sequencing and could see application within this field, providing real-time analysis of newly discovered proteins (Li et al., 2024).

These advances in sequencing technologies have led to an increase in the number of known protein sequences. This can be seen in the explosion in size of the Universal Protein Resource (UniProt) database, with UniProtKB/TrEMBL (unreviewed, automatically annotated) dataset containing approximately 246 million sequence records and the UniProtKB/Swiss-Prot (reviewed, manually annotated) dataset containing approximately 500,000 sequence records as of 2024 (The UniProt Consortium, 2025). Despite this growth in sequencing capabilities, determining the protein's function from its sequence remains complex; experimental methods such as X-Ray crystallography can be used to find the protein's structure, which can in turn be used to determine function. However, these experimental methods can be time-consuming and struggle to keep pace with the influx of protein sequence data (Alberts, 2002). As a result of this, there exists a growing divide between accumulated sequence data, and the relevant protein structure and function information required for real-world applications like drug development.

This sequence-function gap has led to the rise in computational solutions for analysis of protein sequence information and prediction of protein function. Traditional computational methods like BLAST (Basic Local Alignment Search Tool) would compare protein sequences to databases of known sequences, this would infer functionality based on similarity. However, this approach would struggle with sequences with limited similarity to those in the dataset. (Altschul et al., 1990). With the increasing size of available protein sequences, these traditional methods struggle to keep up with the growing scale and complexity; causing researchers to shift towards machine-learning techniques, aiming to capitalize on the large scale of available data.

A novel approach to this is with protein language models. Protein language models borrow techniques from Natural Language Processing (NLP). Bidirectional Encoder Representations from Transformers, or BERT, models learn contextual information of language through self-supervised training on large text corpora. The models take sentence input and translate constituent words into 'tokens', representing each word numerically. These tokens are used to gain contextual understanding based on surrounding tokens. Contextual understanding of language enables NLP models to perform powerful tasks like sentiment analysis and translation (Devlin et al., 2018). Following the success of NLP approaches to understand linguistic context, researchers have explored methods to create protein language models, these instead interpret entire protein sequences as a sentence, tokenizing each individual amino acid within the chain.

These models are then trained on huge datasets of protein sequences from the Universal Protein Resource, learning contextual relationships between amino acids. ProtBERT, one of many models introduced in the ProtTrans paper, uses BERT architecture to model “the language of proteins” (Elnaggar et al., 2022) and will be the focus of this dissertation.

1.2 Summary of the Case Study

To investigate the capabilities of protein language models for function prediction, this dissertation introduces a case study of applying ProtBERT to the binary classification task of predicting whether a given protein sequence is enzymatic or non-enzymatic. This was achieved using a curated subset of the reviewed Swiss-Prot database, where protein sequences were assigned binary labels based on the presence or absence of Enzyme Commission (EC) numbers. ProtBERT, a model containing approximately 420 million parameters and pre-trained on over 216 million protein sequences, was chosen due to its proven capacity for capturing complex sequence relationships. Due to computational restraints caused by local hardware and the size of the model, full fine-tuning was not feasible. Fine-tuning of the model was instead performed using a LoRA approach, a parameter efficient fine-tuning technique that freezes the base layers of the model, reducing training complexity. Model performance is to be evaluated using industry standard metrics; F1-score, precision, recall, and accuracy, assessing how well protein language models understand the contextual ‘language’ of protein sequences. This study aims to demonstrate the capabilities of fine-tuning protein language models for downstream protein function prediction tasks under hardware constraints, offering a scalable approach for the annotation of newly discovered protein sequences.

1.3 Aims and Objectives

The aim of this project is to investigate the effectiveness of fine-tuning protein language models for the binary classification of protein sequences as enzymatic or non-enzymatic. The initial scope of the project intended to explore multi-class hierarchical classification through prediction of full Enzyme Commission (EC) numbers, however, due to computational restraints and the imbalance of EC labels within the dataset a change in approach was taken. This change in scope still provides a meaningful stepping stone for more specific protein function prediction tasks, whilst remaining manageable within the constraints of undergraduate-level resources.

Objectives:

- **Dataset Preparation** Curate a suitable subset from the reviewed Swiss-Prot database, modified to be suitable to the binary classification task. Perform exploratory data analysis and visualise distributions of protein sequences to ensure dataset suitability.
- **Model Selection** Load the base ProtBERT model and perform analysis of trainable parameters, determine the feasibility of fine-tuning within the hardware constraints, estimate compute times with different training configurations.

-
- **Model Fine-Tuning** Use the prepared dataset to fine-tune the model for the binary classification task. Record model parameters and training configurations to ensure reproducibility, allowing for further improvements.
 - **Model Performance Evaluation** Using standard classification performance metrics such as recall and F1-score, assess the model performance at classifying proteins sequences into enzymatic or non-enzymatic. Determine if the model has room to improve with increased training time.
 - **Analysis and Implications** Use the evaluation results from the model to resolve the case study, explore the implications that an accurate model could cause within the field of bioinformatics and proteomic research. Reconsider the feasibility of the original goal of multi-class enzyme classification with sufficient data and computation resources, identify other areas where fine-tuning of protein language models could be relevant.

2. Background Review

In this section of the dissertation, numerous papers, models and tools relevant to the case study will be introduced. It will outline the development of BERT based language models, alongside the changes to apply these NLP techniques to protein language models, specifically Rostlab's ProtBERT. Additionally, the programming libraries and frameworks used to aid model development, evaluation and dataset processing will be covered, alongside the cloud computing environments enabling model training. This background review will provide a foundation for understanding the rationale behind model selection, fine-tuning strategies, dataset preparation, and evaluation methodologies.

2.1 Transformer Architecture

This section will introduce a high-level overview of the progression of Machine Learning (ML) architecture within the field of Natural Language Processing (NLP), finally introducing Transformer architecture, the foundation upon which BERT was built. Broadly, ML is Artificial Intelligence (AI) with the goal of gradual improvements in performance with increased access to data. Neural Networks (NNs) consist of layers of interconnected 'neurons', each layer of neurons transforms the input data through weights, finding the desired weights to assign to these neuron layers for the model to exhibit desired behaviour is the foundation of machine learning (Schmidhuber, 2014). As computers are unable to understand raw text, inputs are encoded into a numerical vector, this allows each node within the hidden layer to process and transform this vector in some manner, with the final output vector being decoded into human-readable form.

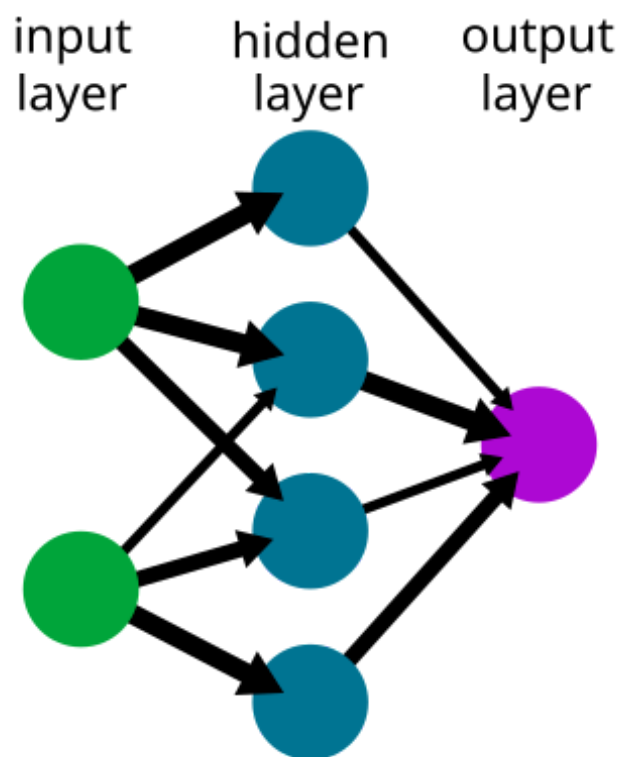


Figure 2.
Simplistic Depiction of a Neural Network
Weights denoted by black lines

Recurrent Neural Networks (RNN) have been used to detect patterns in a sequence of data, primarily text. At a high level, RNNs take in a sequence of input data, processing each input token step-by-step, maintaining a ‘memory’ within their hidden layers. Fundamental problems, however, such as the Vanishing Gradient Problem (VGP), cause RNNs to ‘forget’ earlier input tokens and their relevance during back-propagation. In the context of this dissertation, the longer a protein sequence would be, the less significance the model would place on the starting amino acids within the chain despite their biological importance. To try and solve the VGP, Long Short-Term Memory Units (LSTMs) were used, allowing information to bypass the typical flow of information in an RNN by storing it within gates, effectively learning which information might be relevant later (Schmidt, 2019).

Despite these improvements, for tasks like machine translation, sequence-to-sequence (input sequence to output sequence) models suffered from a bottleneck due to the nature of how inputs were encoded. A source sentence would be encoded into a fixed-length vector which aimed to capture all relevant contextual information regarding that sentence (or protein sequence), which would cause information to be lost in the case of longer sequences. This bottleneck led to the introduction of “Attention”, outlined in the paper “Neural Machine Translation by Jointly Learning to Align and Translate” (Bahdanau et al., 2014). This approach

encodes the input sentence into individual context vectors, each representing a token within the input (word within a sentence). Meaning that during decoding of each token, instead of relying entirely on the fixed context vector generated by the encoder, the decoder can ‘choose’ which input tokens to pay ‘attention’ to.

However, RNN attention models were still dependent on recurrent algorithms to process inputs sequentially, reducing parallelization (capability to process multiple inputs simultaneously) due to each encoding step depending on the previous one. As a result of this, training times were long, and scaling to larger datasets proved computationally inefficient. To overcome these limitations, researchers at Google introduced the Transformer model in the groundbreaking paper “Attention Is All You Need” (Vaswani et al., 2017).

Aiming to tackle issues regarding parallelization, the transformer architecture removes recurrence entirely, focusing on self-attention mechanisms, positional encoding, and multi-head attention. Outlined within the paper, self-attention enables the model to capture relationships between each input token simultaneously without relying on recurrence. To compensate for the lack of inherent positional structure provided by recurrence, positional encoding is used. Positional encoding adds the positional information to each token after encoding. Additionally, multi-head attention enables the model to capture more complex relationships between different parts of the input sequence.

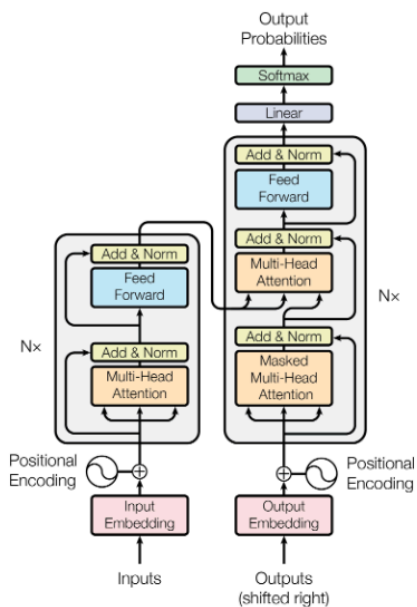


Figure 3.
Depiction of the Transformer Model Architecture
Encoder (Left) and Decoder (Right) Stacks
Reproduced from “Attention Is All You Need” (2017).

Figure 3 taken from the paper shows the transformer model architecture introduced by Vaswani et al. (2017). During training for machine translation tasks, the transformer is trained on sentence pairs containing source tokens and target tokens. In the case of “Attention Is All You Need”, an

example of source/target token data would be pairs of English to French translations, with each source token (an English word) corresponding to a desired target token (a French word). The encoder takes in this source sentence input and simultaneously creates embeddings (vector representations) with contextual information from the self-attention process. The decoder then uses these contextual embeddings to attempt to generate the translated token one token at a time. When predicting the next translated token, the decoder considers both the previously predicted tokens alongside the encoded tokens (with contextual information). Seen in the decoder stack, the masked attention sub-layer prevents the model from using tokens in the target sentence it hasn't "seen" yet, causing a left-to-right translation. The model improves performance by minimizing the loss between what it predicted from the given source sequence, and the actual target sequence. In effect, this creates a similar vector space between English and French words, enabling the transformer to map English input to French output.

For readers interested in more technical details regarding self-attention, cross-entropy loss functions, scaled dot-product attention, and the mathematical functions that comprise these techniques that won't be covered in this dissertation, this has all been thoroughly explained within the paper "Attention Is All You Need".

2.2 BERT Architecture

Transformer architecture in NLP was initially developed to solve sequence-to-sequence tasks, such as translation. However, many NLP tasks, such as sentiment analysis, sentence classification or question answering require architecture that has a deep understanding of a single sequence. One of the most influential models to emerge in this space is BERT, introduced in the paper "BERT: Pre-training of Deep Bidirectional Transformers For Language Understanding" (Devlin et al., 2018). Unlike the encoder-decoder stacks used within the transformer architecture shown in figure 3, BERT uses only the encoders; this enables the BERT model to generate contextualized embeddings for each token using Masked Language Modelling techniques.

Outlined in section 3.1 'Pre-training BERT' Task #1, some percentage of the input tokens are masked at random, then these tokens are predicted. MLM allows the model to learn bidirectional context by predicting masked tokens based on the other tokens in the sequence, in both the left and right directions. This context awareness assigned to each token within a sequence means that the model can distinguish between two identical input tokens or homonyms. For example, in the sequences, 'He was right' and 'He was right there', BERT can recognise that the specific meaning of the word 'right' changes with surrounding context.

Following this, Section 3.2, Task #2 Next Sentence Prediction (NSP) details the steps to pre-train BERT to understand relationships between sentences. This is done through binary labelling "Specifically, when choosing the sentences A and B for each pre-training example, 50% of the time B is the actual next sentence that follows A (labelled as IsNext), and 50% of the time it is a random sentence from the corpus (labelled as NotNext)." NSP helps BERT to learn inter-sentence dependencies and logical coherence across text segments.

The strengths of the BERT model are further highlighted within Section 3.2, Fine-Tuning BERT. It is shown that fine-tuning the BERT model for downstream tasks is relatively inexpensive computationally, this makes it suitable for undergraduate level research projects which are constrained by computational resources. Appendix A.3 of the BERT paper provides hyperparameter value ranges with success across a wide array of fine-tuning tasks. This gives a strong foundation from which to generate and evaluate new models based on BERT and its derivatives, such as ProtBERT.

2.3 ProtBERT

This section will cover ProtBERT, the protein Language Model (pLM) introduced within “ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning” (Elnaggar et al., 2021). It builds upon the architecture of BERT, adapting the self-supervised MLM approach to learn contextual representations of amino acid sequences.

In contrast to NLP, where tokens are typically words within an input sentence, ProtBERT treats protein sequences as the input sentence, tokenizing each constituent amino acid. This enables the pLM to capture structural patterns within proteins through self-supervised training on very large protein databases. ProtBERT specifically was trained on UniRef100, containing 216 million protein sequences with 88 billion total amino acids (Section 2.1 Data for Protein Language Models).

Adapting the BERT approach to generate highly contextual embeddings for each amino acid residue makes ProtBERT highly suitable for downstream bioinformatics tasks, such as the focus of this dissertation, enzyme classification. By fine-tuning the model using labelled protein datasets, it becomes possible to leverage supervised learning against the base ProtBERT model’s deep contextual understanding.

This dissertation will use the base ProtBERT model as a foundation, applying fine-tuning techniques to adapt the model for the binary classification task of identifying enzymatic protein sequences, seeking to demonstrate practical applications of protein language models within the field of bioinformatics.

2.4 Tools and Technologies:

In the implementation of this project, numerous python libraries were used to aid in dataset preprocessing, model training and performance evaluation. These libraries are widely adopted within the machine learning community, providing a freely available, reproducible environment for experimentation with different downstream model tasks. Extensive documentation is freely available for each tool used.

2.4.1 Data Analysis and Preparation

2.4.1.1 Pandas

Python Data Analysis Library, used to load and modify tabular datasets, such as Swiss-Prot. Enables exploratory data analysis on large protein datasets, feature selection for classification tasks.

2.4.1.2 Matplotlib

Python library for data visualisation for the creation of visual diagrams showing dataset distribution. Helps to aid in early data preprocessing, ensuring dataset distribution retains relevant information.

2.4.2 Model Development

2.4.2.1 Hugging Face Transformers

HuggingFace’s Transformers library (Wolf et al., 2019), open source library providing tools for downloading models and performing fine-tuning tasks. In this case, provides both the base ProtBERT model alongside the tokenizer used.

2.4.2.2 Hugging Face Community Hub

(Wolf et al., 2019) An online resource for streamlined downloading and sharing of fine-tuned models and prepared datasets. Enables consistent, reproducible training by providing open-source access to the resources used. Particularly relevant to this project as it allows models to be saved to the hub during training epochs, this saves the model periodically during training and serves as a backup to reload the model in the case of system failure during training (e.g. Google Colab compute resources expire, terminating project runtime).

2.4.2.3 Hugging Face Datasets

Streamlines loading of large datasets into the local compute environment and the tokenization process. Pre-tokenized datasets can also be uploaded to the hub, allowing for each model notebook to save compute resources spent on tokenization to use within the training pipeline. Supports pre-splitting of training datasets into a DatasetDict structure, storing ‘train’, ‘test’, and ‘validation’ splits for reproducible training data.

2.4.2.4 Hugging Face PEFT

Parameter Efficient Fine-Tuning library is an extension of transformers, used to apply LoRA techniques. Supports adjusting LoRA hyperparameters and freezing of base ProtBERT layers.

2.4.3 Model Training

2.4.3.1 Google Colaboratory

Google Colab provides cloud-computing access, allowing for training of large models with freely available GPU hardware (T4 GPU) reducing constraints imposed by local hardware. It should be noted however, whilst Colab does provide a run-time free of charge, when training models with a high number of iterations (based primarily on the size of training data) it becomes trivial to

exceed the allocated number of compute units provided. This leads to a soft-cap on the amount of training data you can feed into a model, shown in the methodology section.

2.4.3.2 Local Hardware Environment

Initial dataset analysis and processing were performed on a local machine with the following specifications:

- OS: Debian 12
- CPU: Intel(R) Core(TM) i7-1065G7 @ 1.30GHz
- RAM: 4GB on board + DRAM DDR4 4GB
- GPU: Intel Corporation Iris Plus Graphics G7 (Integrated Graphics)

Due to constraints imposed by the lack of a dedicated graphics card, all model training was performed using Google's cloud computing solution.

2.5. Enzyme Commission (EC) Numbers

This section will broadly cover the concept of EC numbers as it is relevant to the project classification task. Within 'Enzyme Nomenclature' (<https://iubmb.qmul.ac.uk/enzyme/>) the naming conventions for enzymes are highlighted. Relevance to the project is the nature of how enzyme commission numbers are created, EC numbers specify enzyme-catalysed reactions as opposed to enzymes as a whole. This means that proteins can have numerous EC numbers dependent on their catalytic function, standing to reason that multiple protein sequences can have the same EC numbers. Classification is handled hierarchically, with 4 classes in the form 'W.X.Y.Z'. Each progressive subclass represents a finer classification of the enzyme.

3. Methodology

This section of the dissertation will provide an in-depth technical overview of the methodologies used to fine-tune a BERT based Protein Language Model (ProtBERT) for the task of binary classification task of predicting input protein sequences as enzymatic or non-enzymatic. Processes involving dataset collection, analysis, feature extraction and preparation will be covered, alongside an introduction to the Hugging Face libraries used for model training and evaluation. Difficulties regarding dataset complexity and computational constraints will be discussed, motivating the change in project scope.

3.1. Dataset Acquisition

The dataset used in this project was sourced from the Universal Protein Resource (UniProt) (The UniProt Consortium, 2024). Due to the nature of fine-tuning tasks on BERT based language models requiring supervised training with labelled data (Devlin et al., 2018) the Swiss-Prot reviewed subset of the UniProt Knowledgebase (UniProtKB) was chosen for the classification task. UniProt provides a comprehensive database consisting of protein sequences alongside functional data and structural data, including EC numbers. The Swiss-Prot subset includes all protein sequences that have been manually reviewed to ensure feature accuracy, aiming to avoid the “feedback loop” problem (Hari. 2015). Feedback loops occur when prediction models use their own predicted outputs as training data, causing biases to propagate throughout the system, becoming increasingly detached from real-world data.

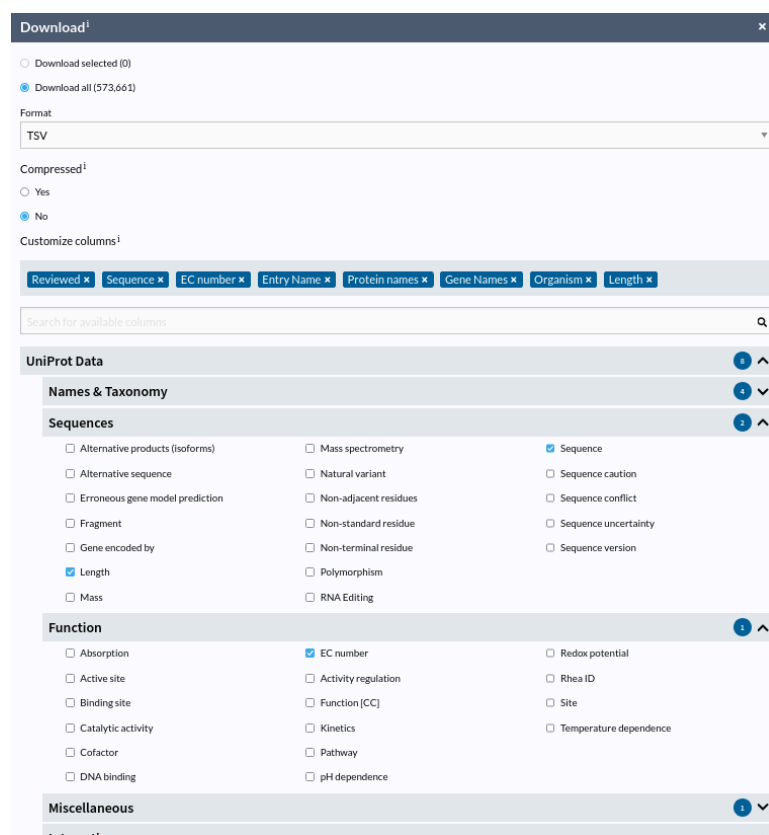


Figure 4.
 Screenshot of the [UniProt.org](https://www.uniprot.org) Website
 Download Interface Showing Available Feature Selections.

Figure 4 depicts a screenshot of the UniProt website, shown when you attempt to download a dataset. As shown, curating the dataset to required project needs is straightforward; users can select required features from the dropdown menu and download the data in their desired format.

For this project, the chosen features were ‘Sequence’ and ‘EC number’, as these are the essential fields required for the enzyme classification task. ‘Sequence’ serves as the input for the ProtBERT model, and the EC number will be processed into a trainable target label. The dataset was downloaded in the TSV (Tab-Separated Values) format, this was due to the Pandas libraries compatibility with this file type.

3.2. Dataset Analysis and Preparation

Preliminary Data Analysis was performed through the pandas and Matplotlib libraries within a Visual Studio Code notebook environment running on local hardware. This analysis included assessing dataset size, EC number distribution and sequence length distribution. As a starting point, the dataset holds 573,661 entries, with 278,804 containing EC numbers, and 294,857 without.

From analysis of the dataset, it is clear that Enzyme Commission numbers are hierarchical in nature, with 4 classes. The primary class number ranges from 1 through 7, with

each proceeding subclass containing a range of numerical values (McDonald et al., 2009). To visualize this data, a sunburst chart was implemented from the plotly library; due to the volume of unique EC numbers when accounting for each level, this chart was limited to display the first two counts of EC values.

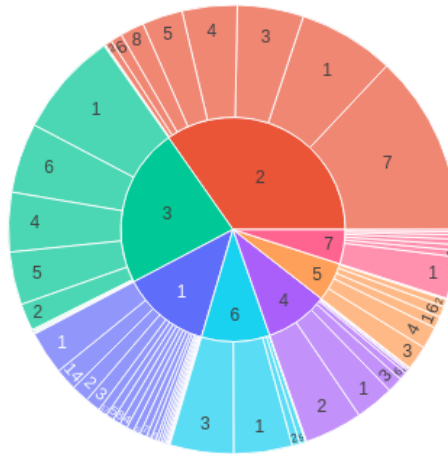


Figure 5.
Sunburst Chart showing EC Number Distribution For 2 Classification Levels
Implementation Provided in Appendix A

Reviewing Figure 5 highlights that the complexity of an EC number classification task increases with the addition of each subclass. At 2 classification levels, there are 149 unique numbers within the dataset, the full 4-level classification task results in over 7,000 targets for a classification challenge. As a result of this, the project scope was changed from multi-classification of EC numbers based on protein sequence, to a binary classification task to predict a binary label “is enzymatic”.

Reflecting this change in scope, the dataset was modified to contain two columns ‘Sequence’ and ‘is_enzymatic’. Target label ‘is_enzymatic’ was determined based on whether the ‘EC number’ field is null (0) or contains information (1). Next, sequence length distribution was analysed. Basic sequence length analysis gathering the maximum and minimum lengths returned the shortest and longest proteins within the dataset at a size of 2 and 35,213 amino acids, respectively. Researching how ProtBERT (Elnaggar et al., 2022) was pre-trained, section 2.4 of the ProtTrans paper states “ProtBert was first trained for 300k steps on sequences with a maximum length of 512 and then for another 100k steps on sequences with a length of a maximum length of 2k.”. This prompted the removal of proteins with long sequences, after culling proteins with amino acid chains larger than 4,000 a length distribution histogram was created.

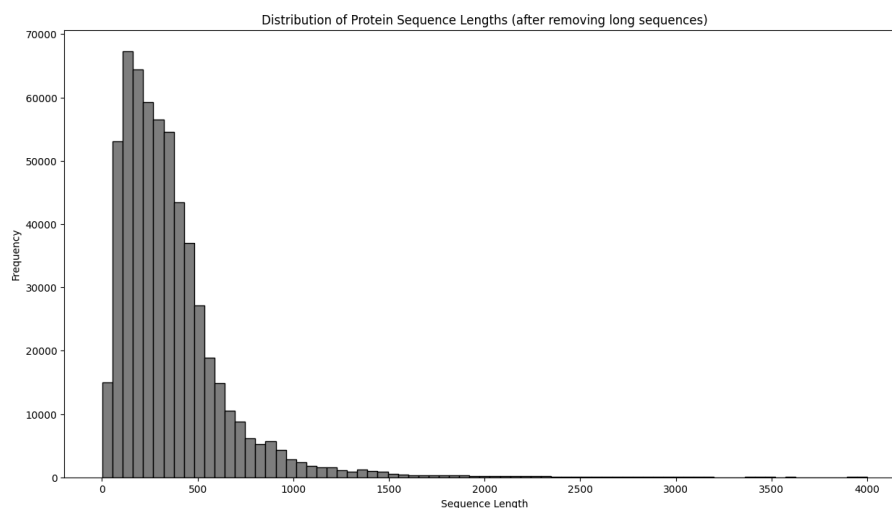


Figure 6.
Histogram showing the Frequency Distribution of Sequence Lengths

While the tokenizer and ProtBERT model can technically handle longer sequences, the heavy right-skew of the distribution indicates the presence of few extremely long proteins. In the interest of reducing model training times, a final max sequence length for the dataset was established at 1024.

Following sequence length filtering, the dataset was assessed to find duplicate entries that should be removed before model training to prevent data leakage. Data leakage refers to the concept of machine learning models training on information it shouldn't have access to (Kaufman et al., 2011). In this context, if duplicate protein sequences are present then during separation into training, testing and validation splits, identical sequences could be found within each set. The number of duplicate sequences within the dataset was found to be 87,750, of these 87,750 only 22 were shown to have contradictory `is_enzymatic` labels. These contradictory labels were removed from the dataset, the rest of the duplicated sequences were deduplicated. The resulting size of the dataset after this process is 467,704 entries.

Counts of each '`is_enzymatic`' label were generated to ensure that the training data holds a moderately balanced class distribution between enzymatic and non-enzymatic sequences, this is to prevent the introduction of bias in binary classification tasks. Of the 467,704 entries, 228,224 are classified as enzymatic, the resulting 239,480 are classified as non-enzymatic. This means there is a 48.8% to 51.2% split between label values.

Satisfied with this split for the prediction task, the final step of pre-processing involves separating each amino acid within a sequence by whitespace, this is in line with the ProtBERT tokenizer's expected input specifications. The preprocessed dataset was then split into training,

testing and validation sets (70/15/15) to ensure consistent, reproducible results. The pandas dataframe was then converted into a Hugging Face DatasetDict object and pushed to the hub, enabling the dataset to be readily available in my cloud computing environment.

3.3. Tokenization and Input Encoding

Before fine-tuning can begin, input protein sequences must be tokenized. For this, the AutoTokenizer class from the Hugging Face Transformers library was used to load the tokenizer associated with the publically available ProtBERT model sourced from “roslabs/prot_bert” (Elnaggar et al., 2022). This tokenizer splits amino acid sequences into individual characters with numerical representations.

The tokenizer takes an input of protein sequence and target label, ‘is_enzymatic’ transcribing them into 3 new fields for the dataset; ‘input_ids’, ‘attention_mask’ and ‘token_type_ids’ each representing a set of numerical values the chosen model, ProtBERT, understands.

‘Input_ids’: A set containing the numerical representation of each word (in the case of NLP tasks) or amino acid embedding. These are assigned according to the tokenizer’s configurations, which can be found in the file ‘tokenizer.json’ once the model has been downloaded. Meta tokens are also defined, such as [CLS] or ‘classification’ which signals the start of a new sequence to the model; due to this, every ‘input_ids’ set begins with the number 2.

‘Attention_mask’: Attention denotes how much the model should attend to a specific embedding. In this tokenizer, it is used to tell the model to ‘ignore’ irrelevant information. Due to the assigned max length of 1024, sequences shorter than 1024 amino acids will be padded to this size and the attention mask for padded tokens will be set to 0.

‘Token_type_ids’: Distinguishes input sequence for tasks requiring two sentence inputs, such as Next Sentence Prediction covered within section 2.2 of this paper. The nature of this single sequence classification task means that this field will not be relevant to the project, but it is still generated by the tokenizer for compatibility with BERT models that expect this field.

It is worth noting that when working with pre-trained models for downstream tasks, the configuration settings should be inspected to ensure it is compatible with your use case. In this project, the model_max_length tokenizer parameter was manually set to 1024 to limit memory usage and conserve training time.

For example code in the following implementation, visit Appendix B. To tokenize the entirety of the preprocessed dataset (467,704 entries) the data was first loaded into local memory using the Hugging Face Hub API, accessing a personal repository through the load_dataset function. The roslab_protbert model and tokenizer were also loaded into local memory, parameters of the tokenizer were adjusted to prevent excess padding. A tokenization function was defined to enable batching, a process in which tokenization time is reduced by enabling

multiple sequences to be processed simultaneously. The time taken to tokenize the full dataset within the Google Colab runtime (T4 GPU) was 5 minutes and 30 seconds. In the broader context of this project, efficient usage of Colab compute units is paramount; using the aforementioned Hugging Face API's this tokenized dataset was uploaded to the repository for ease of access in dedicated training scripts. The resulting structure of the dataset is shown in Figure 7.

Sequence	is_enzymatic	input_ids	token_type_ids	attention_mask	label
MAAPVSEPTVAR QKLLALLGQVQ_	0	[2, 21, 6, 6, 16, 8, 10, 9, 16, 15, ...]	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...]	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...]	0
MGLCCGSDRVVT KYSKEDIQQPT_	0	[2, 21, 7, 5, 23, 23, 7, 10, 15, 14, ...]	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...]	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...]	0
MKLAACFLTLLP GFAVAASWTSP_	1	[2, 21, 12, 5, 6, 6, 23, 19, 5, 15, ...]	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...]	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...]	1
NLPPLWTLSSLG AVAGKEVCYER_	1	[2, 21, 5, 16, 5, 24, 15, 5, 10, 5, ...]	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...]	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...]	1
MTSTPIIPADLL PADGRFGCGPS_	1	[2, 21, 15, 10, 15, 16, 11, 11, ...]	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...]	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...]	1

Figure 7.
Screenshot of the Hugging Face Repository to Highlight Data Structure

3.4. Model Training and LoRA Fine-Tuning Pipeline

To classify protein sequences as enzymatic or non-enzymatic in nature, this study aims to fine-tune the ProtBERT model sourced from ‘roslab/prot_bert’. Full fine-tuning of the ProtBERT model was attempted initially, though was deemed computationally infeasible due to both the size of the dataset (470,000 entries) and the total number of trainable parameters within the ProtBERT model (420 million). Given the scale of the model, alternative approaches to fine-tuning were considered. A parameter efficient fine-tuning method called Low-Rank Adaptation was used. Proposed in the paper “LoRA: Low-Rank Adaptation of Large Language Models.” (Hu et al., 2022), LoRA emerges as a potential solution to the problem of full pre-training becoming too computationally expensive as larger language models are introduced. Full fine-tuning of a model aims to retrain all parameters, whereas the LoRA method freezes these base weight parameters, injecting trainable rank-decomposition matrices into layers such as the self-attention mechanism (outlined in section 2.1 of this dissertation). These matrices drastically reduce the number of trainable parameters whilst maintaining comparable model performance.

When attempting to deploy training pipelines, it is understood that the number of iterations required for model training scales proportionally with the amount of training data.

Investigating full fine-tuning on local hardware with the complete dataset, the number of iterations was 245,544. At a training rate of over 300 seconds per iteration, this equated to over 2 years of continuous model training.

To facilitate faster model deployment, 1% of the original dataset has been selected at random. Steps were taken to ensure this subset kept a near-even distribution between classification labels (training split: 1608/1665; testing split: 333/368; validation split: 346/355; enzymatic/-nonenzymatic samples). In the aforementioned interests of reproducibility and consistency, this shortened dataset has been uploaded to the Hub; this link can be found in Appendix C.

Whilst the reduced dataset does limit the performance that can be obtained, the methodology used in training and evaluating the models can be freely extended to larger datasets given more time and computational resources. The objective of this implementation is to demonstrate the feasibility of protein function classification through fine-tuning of protein language models alongside the accessibility afforded by the Hugging Face Transformers library.

The binary classification head is added to the ProtBERT model through the example code “protbert_binary_classifier = AutoModelForSequenceClassification.from_pretrained(“roslab/prot_bert”, num_labels=2)”. This loads the pretrained model with a sequence classification head with an output size set to 2, representing our ‘is_enzymatic’ binary labels.

A baseline TrainingArguments class was defined through the Transformers library. In this section the arguments chosen will be introduced, these parameters will remain consistent between training both the fully fine-tuned ProtBERT model and the Low-Ranking Adaption of the model. An evaluation of the performance between these two models will be presented in the results section of this dissertation. Explanations for each parameter function will be paraphrased from the Hugging Face Documentation.

Key Parameters Chosen for Training Arguments:

Parameter Name	Chosen Value	Optional Values	Explanation	Justification
per_device_train_batch_size	8	[1, 2, 4, 8, ...]	Batch sizing using GPUs parallel processing power. Higher values speed up computation time at the cost of more required VRAM.	Experimenting with different batch sizes, 16 causes cuda out of memory errors, the best available batch size for our Colab environment was chosen.
per_device_eval_batch_size	8	[1, 2, 4, 8, ...]	As above, instead affecting evaluation times.	As above.
fp16	True	[True, False]	Toggles	Reduces complexity of

			mixed-precision training, where ‘fp’ refers to floating point precision. Computations performed with a mix of fp16 and fp32.	calculations, reducing memory consumption, preserving GPU RAM.
num_train_epochs	5	[1, 2, 3, 4, 5, ...]	Sets the total number of training epochs, where each epoch is a single pass-through the entire training dataset.	Provides enough data points to represent within the results chart, showing how model performance changes during each epoch. Enables analysis of underfitting/overfitting.
learning_rate	5e-5	[5e-5, 3e-5, 2e-5, 4e-5, ...]	Defines how quickly the model adjusts internal parameters in response to error made during each training step, affecting training speed and generalization (Wilson et al.)	Relatively small value to avoid drastically changing pre-trained model weights. This learning rate can be revisited in the future through experimentation of different values.
weight_decay	0.01	[0, 0.001, 0.002, ...]	Constrains the complexity of large neural networks by limiting the growth of weights (Krogh et al., 1991)	Commonly used weight decay value, chosen in particular due to presence in the BERT paper.
metric_for_best_model	“f1”	[“f1”, “recall”, “accuracy”, “precision”, custom]	Defines a metric to use to determine the best model at each save strategy, epoch in this case	F1-score combines ‘recall’ and ‘precision’ scores, High F1-score indicates high rates of identifying enzymatic proteins, and low rates of misclassifying non-enzymatic proteins.

Table 1.

More training parameters than the ones mentioned in Table 1 were used during model training, but the important parameter settings for reproducibility have been highlighted. One parameter, ‘push_to_hub=True’ stood out as particularly noteworthy. With this setting enabled, the model’s tokenizer, configurations, and training logs are automatically uploaded to the Hugging Face Hub at the end of each epoch. This proved especially valuable during full fine-tuning of the ProtBERT model; limitations in VRAM caused the training to conclude early, just after the fourth epoch. Due to this automated checkpointing, training could have seamlessly been resumed

from the most recent save state, however the checkpoint directory, or “output_dir” was set to local Colab memory instead of being pushed to the hub alongside the rest of the model configs.

In the broader academic context, this automatic checkpointing system encourages reproducibility and transparency in machine learning research. Models can be evaluated at each training stage due to the version control inherent within the git-based hugging face hub; this contributes to the open-source nature of the field, enabling anyone to expand upon previous work with limited computational resources.

Once suitable training arguments have been defined, the next step is to build the trainer. Conveniently, Hugging Face also abstracts this process, using the ‘Trainer’ function imported from the ‘Transformers’ library. Sample code will be shown in Figure 8.

```
trainer = Trainer(  
    model=protbert_binary_classifier,  
    args=training_args,  
    train_dataset=dataset["train"],  
    eval_dataset=dataset["eval"],  
    tokenizer=tokenizer,  
    compute_metrics=compute_metrics,  
    data_collator=data_collator  
)  
  
trainer.train()
```

Figure 8.

Sample Code of the Trainer

https://huggingface.co/docs/transformers/en/main_classes/trainer

Compute metrics generate ‘f1’, ‘precision’, ‘accuracy’ and ‘recall’ metrics to monitor performance during evaluation. These metrics are obtained using the model’s raw predictions (logits) against the actual target (label) through the validation dataset.

Implementation of the LoRA approach retains these outlined steps. After adding the classification head to the base model, the ‘peft’ library is used to freeze the base layers.

```
# docs: https://huggingface.co/docs/peft/en/package_reference/lora  
from peft import get_peft_model, LoraConfig, TaskType  
# Set up LoRA  
lora_config = LoraConfig(  
    r=8,  
    lora_alpha=16,  
    target_modules=["query", "value"],  
    lora_dropout=0.1,  
    bias="none",  
    task_type=TaskType.SEQ_CLS # Sequence classification due to input being tokenized protein sequences  
)  
  
model = get_peft_model(protbert_binary_classifier, lora_config)  
  
# Inspect trainable params  
model.print_trainable_parameters()
```

Figure 9.

Sample Code of LoRA Configuration

https://huggingface.co/docs/peft/en/package_reference/lora

For further information on each LoraConfig parameter, it is advised to read the documentation. For a high-level overview, see Table 2.

Key Parameters Chosen For Lora Configuration

Parameter Name	Chosen Value	Optional Values	Explanation	Justification
r	8	[2, 4, 8, 16, 32]	Defines rank of injected update matrices. Lower ranks result in fewer trainable parameters.	Maintains a balance between training time and retaining enough trainable layers for the model to learn. A baseline value to prototype the LoRA approach taken from the literature.
lora_alpha	16			Typically set to 2 x r. Another baseline value.
target_modules	["query", "value"]	["query"], ["value"], ["key"] and ensuing combinations	The modules (for example, attention blocks) to apply the LoRA update matrices	Adapting only query and key values seeks to utilise the contextual understanding gained from ProtBERT's pretraining.
task_type	SEQ_CLS	[SEQ_2_SEQ_LM, CasualLM, TOKEN_CLS, ...]	Assign the specific target task to the LoraConfig	Sequence classification chosen as we're attempting to classify (is_enzymatic?) a target sequence (amino acid chain).

Table 2.

After applying the LoRA configuration, using the built-in function ".print_trainable_parameters()" displays the number of trainable parameters as 985,090, 0.234% of the ProtBERT model's 420,918,276.

The LoRA adapted model is then trained the same way as full fine-tuning, using the Trainer class shown in Figure 8. It is important to note that LoRA wrapping changes the way in which users upload their model to the hub, the `push_to_hub` method only uploads the base model with changes in model weights.

4. Results

In this section, four models trained to classify protein sequences as enzymatic and non-enzymatic are introduced as project outputs. These models consist of two variants of ProtBERT, one utilizing a Low-Rank Adaptation (LoRA) approach and another using full fine-tuning. Each fine-tuning approach was trained on two different dataset sizes, 1% and .25% of the total database, split as follows:

- 1% Dataset split: 3,270 training samples; 701 validation samples; 701 Testing samples.
- .25% Dataset split: 818 training samples, 175 validation samples, and 175 testing samples

The models:

- Model A, Full fine-tuning using .25% of the dataset
- Model B, LoRA fine-tuning using .25% of the dataset

- Model C, Full fine-tuning using 1% of the dataset
- Model D, LoRA fine-tuning using 1% of the dataset.

Model performance for the binary enzyme classification task was recorded for each training epoch, their performance will be evaluated on industry standard classification metrics; F1-score, recall, precision, and accuracy calculated on the validation set after each training epoch.

Typically data is split into 3 sections, the training, validation, and testing split, conventionally 70/15/15. The training portion allows the model to learn inference between target input and output, in this case protein sequence to “is_enzymatic” label, aiming to minimize a loss function, once the training data has been wholly ‘consumed’ this signifies the end of an epoch. In multiple-epoch training, the model is then evaluated against the validation set. This provides a way to monitor model performance during training. By introducing unseen data the model can be assessed for under or overfitting, adjusting hyperparameters to optimize the model to best generalize to new data. The test set contains the final portion of unseen data, to test how well the model generalizes to completely unknown data. This delivers the most accurate performance metrics for assessing model generalization.

Despite creating the separate test set in a prior section (see Figure 7.), persistent issues regarding loading models from the hub coinciding with time constraints and run-time disconnections prevented this final model evaluation from being carried out. Due to this, all evaluation metrics in this section will be derived from validation performances during the training process, screenshots of which can be found in Appendix D.

The absence of evaluation on the testing set
Training time will also be considered, measuring the tradeoff between performance improvements and computational cost.

4.1. Training Times

For each model, `per_device_train_batch_size` was set to 8. This means that the total number of iterations required for fine-tuning is consistent, this section will explore how LoRA affects iteration speeds. This information is gathered from the Trainer output at the end of a training cycle.

Model	Total Training Steps	Train Steps Per Second	Total Runtime (s)
Model A (full .25%)	515	0.309	1,664.04
Model B (LoRA .25%)	515	0.695	741.4
Model C (full 1%)	2050	UNK	UNK
Model D (LoRA 1%)	2050	0.678	3,022.82

Table 3.

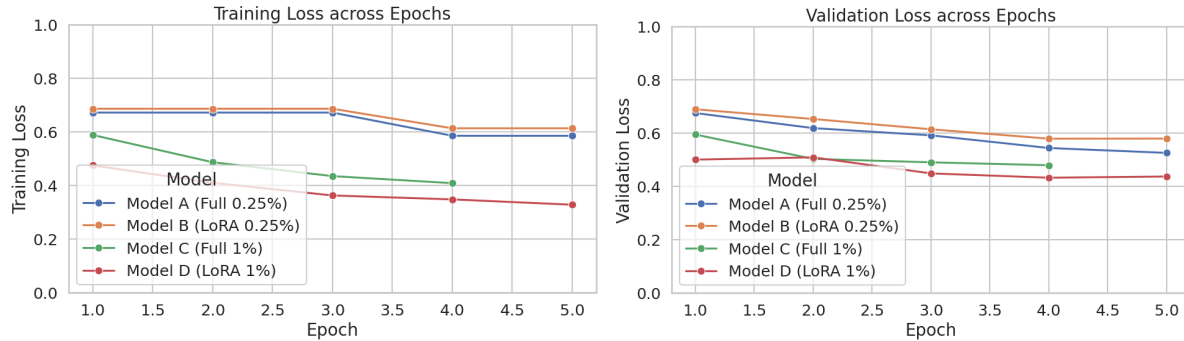
Analysis of Training Times:

LoRA fine-tuning models (B and D) consistently showed faster iteration speeds when compared to full fine-tuning models (A and C). Most notably, model B has a training step speed of .695 steps per second, boasting training times over twice as fast as the equivalent model with full fine-tuning implementation. This demonstrates the computational efficiency of LoRA fine-tuning over the traditional fine-tuning approach.

Comparing the total training time between models B and D, a growth in input data of 300% corresponded to an 307.7% increase in training time. This proportional increase highlights the scalability of the LoRA framework, making it well suited for larger training tasks. Further analysis is required to examine potential tradeoffs between model performance and reduced training times, if any.

4.2 Validation Performance During Training

4.2.1 Training and Validation Loss Trends



Figures 10 & 11

Figures 10 & 11 display training and validation loss curves for each model. For the .25% dataset models, training loss flatlines at around 0.7, this suggests that the models failed to improve during training, possibly due to the small dataset size or limited number of training epochs. Contrasting this, the 1% models see marked reduction in training loss over the first few epochs, eventually appearing to plateau between 4 and 5.

Regarding validation loss, no model shows significant improvement over 5 epochs, signifying difficulty in generalizing to unseen data. This could be improved through expansion of dataset size, providing more examples for the models to learn from.

4.2.2 Evaluation Metric Trends

Each model was evaluated against the same validation set for each of the 5 epochs, validation scores for these metrics are shown in Figures 12 through 15. Final epoch evaluation of Model D is absent, due to restrictions on kernel runtime in the cloud-computing environment.

For binary classification tasks, each predicted label has four possible outcomes:

- True Positive (TP), the model accurately predicts a positive label (1).

- True Negative (TN), the model correctly predicts a negative label (0).
- False Positive (FP), a negative value is incorrectly predicted to be positive.
- False Negative (FN), a positive value is predicted to be negative.

The counts of each of these outcomes form the basis of the evaluation metrics used within this section (Rainio et al., 2024). These metrics include:

- Recall
- Precision
- Accuracy
- F1-Score

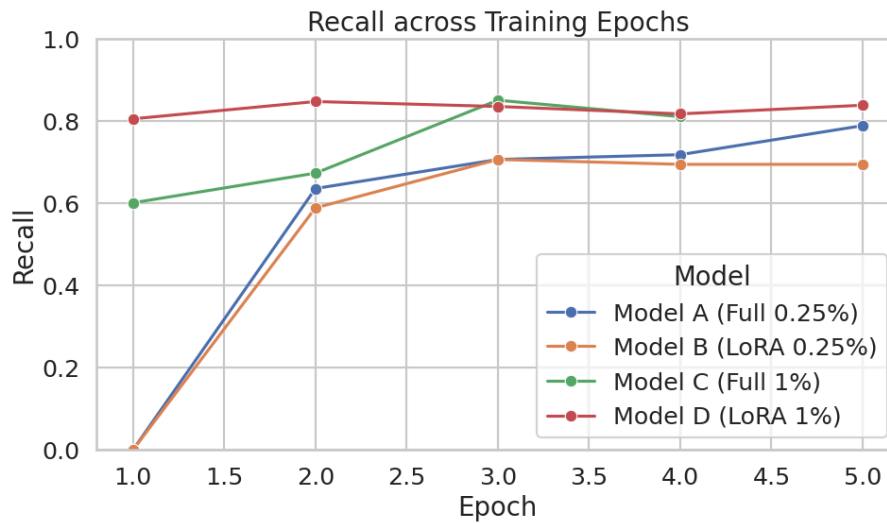


Figure 12.

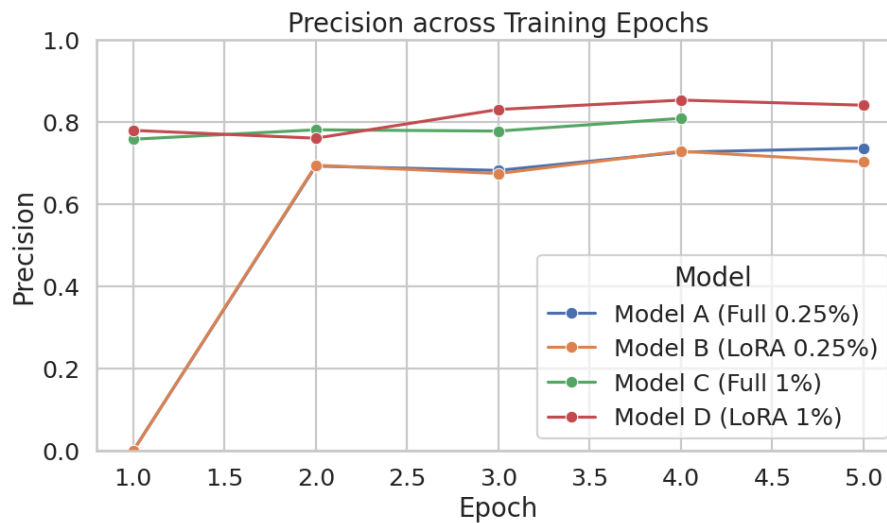


Figure 13.

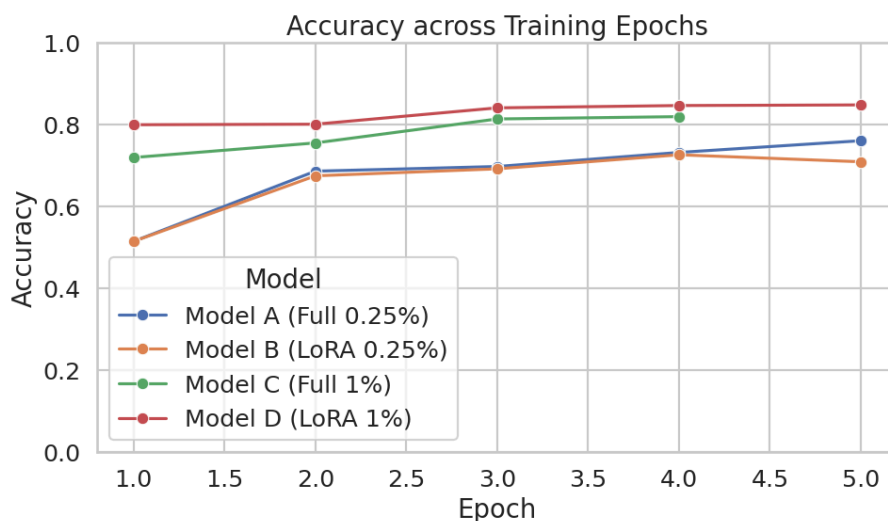


Figure 14.

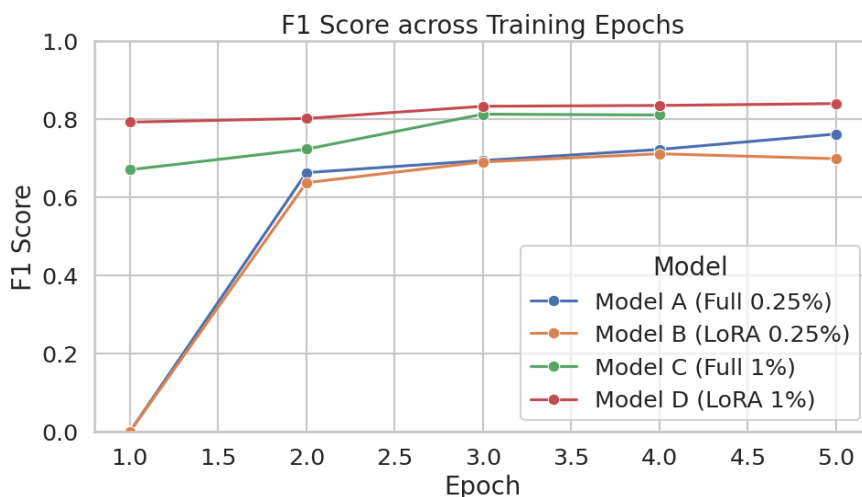


Figure 15.

The trends within each metric provide insights into how each model performed across 5 epochs. This section will outline what each metric represents.

- Recall measures the model's ability to correctly label all enzymatic (positive) protein sequences. Defined as $TP / (TP + FP)$, higher recall implies the model is less likely to miss enzymatic sequences. Shown in Figure 12, the best performing model with regards to recall is Model D, capturing more than 80% of the positive label. In contrast, the worst performing model in this domain is Model B, the other LoRA implementation. This

highlights the importance of training data volume in prediction tasks. However, recall isn't a valuable metric on its own, a model could simply predict every sequence to be enzymatic and achieve a high recall score.

- Accuracy is calculated through $(TP + TN) / (TP + TN + FP + FN)$, where minimizing the number of total false predictions improves the score. This metric is particularly relevant in this case due to the even class distribution of target labels and serves as a suitable metric for comparing model performance. Shown in Figure 14, Model D outperforms the rest of the field.

An insight worth mentioning, is that at the fourth training epoch (last available datapoint for Model C) for each metric, including training/validation loss, the best performing model is model D (LoRA 1%), followed by Model C (Full 1%), then Model A (Full .25%), and finally, Model B (LoRA .25%). This raises an interesting question, why does LoRA lose out in performance to full fine-tuning for the smaller dataset, but not the larger one? Searching for an answer to this question led me to the paper “LoRA Learns Less and Forgets Less” (Biderman et al., 2024). In which it's stated “We find that with commonly used low-rank settings, LoRA substantially underperforms full finetuning, while typically requiring longer training”, which conflicts with our results shown above. However, the following sentence “Despite LoRA's limitations, we show that it consistently maintains better source-domain performance compared to full fine tuning” helps to answer our question. From this research I believe that Model D performs the best due to relying most heavily on the encoded information within ProtBERTs parameters. This would explain why it scores highly at the first epoch and fluctuates at that value; there is not enough supervised data within the 1% subset to improve upon ProtBERTs source domain performance.

4.4 Implications Of Results

With all 4 models trending towards accuracy rates of 70%+, with Model D in particular boasting 80% prediction accuracy by its first epoch, these results demonstrate ProtBERTs intuitive knowledge in the protein space. With strict hardware limitations imposed and minimal training time, generalizable models for the enzyme classification task were created, this project provides an overview of this process. With iterative development afforded by the open-source nature of the Hugging Face Hub and libraries, fine-tuning protein language models for downstream function prediction tasks has the potential to revolutionize the bioinformatics space. Conceptually, with machine learning models aiding researchers in the functional annotation of proteins, this will create a positive feedback loop, where AI output quickly becomes real, verifiable data that can be used to train future models. This is opposed to the traditional feedback loop as introduced in section 3.1 of this paper.

4.5 Considerations For Improved Model Evaluation Data

This section will briefly consider methods to improve model quality and collection of evaluation data whilst still considering hardware constraints.

Define custom pipelines to save and load models at specified epochs. By saving models during the training process, GPU cache can be freely cleared, and the latest iteration loaded. Increasing the number of training epochs in the charts above would improve readability. The version control inherent within Hugging Face could enable parallelization of both training and evaluation. As each previous epoch is stored to the hub, it can be evaluated on the held out test set for accurate performance metrics without introducing data leakage.

Further reducing maximum sequence length for truncation, could compare model performance between those with access to more sequences against those with fewer but more detailed sequences.

5. Evaluation

In this chapter the project objectives will be covered sequentially, focusing on how well each was executed.

5.1. Dataset Preparation

A key learning objective for this project was to curate a suitable dataset for the downstream fine-tuning task.

This objective was successfully achieved; exploratory data analysis techniques were applied to uncover anomalous records, such as duplicate sequences with contradictory target labels. Duplicate sequences were removed to prevent data-leakage between training, test and validation sets. Extremely long sequences were filtered from the dataset, and classification labels were derived from reviewed sources.

Visualisations highlighted the complexity inherent within EC number prediction, motivating a change in scope to binary classification; feature engineering converted these complex EC labels into more manageable binary labels.

Despite working with subsets of the available data (.25%, 1%) for this project's implementations, the dataset is readily accessible. However, considerations could have been made when encountering numerous difficulties with model implementation to revisit the dataset and truncate sequence lengths further for higher batching counts during training.

5.2. Model Selection

This goal was partially met, the ProtBERT model was correctly initialised within numerous environments, however, analysis of the model defaulted to reading the introductory paper "ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning",

which outlines how the model was pre-trained, as well as tokenizer embeddings. Examining the feasibility of fine-tuning was not accomplished, nor was estimating computation times for model training.

5.3. Model Fine-Tuning

Multiple fine-tuning implementations according to different specifications were generated, outlined in Section 4. Full fine-tuning models were uploaded to the Hugging Face hub. It should be stated that the LoRA adaptation layers were not correctly saved to the hub, only the base model weights were uploaded. Whilst technically the stated objective was met with the fully fine-tuned models, ensuring the LoRA models were pushed with their adaption layers would enable more accurate representation of the model capabilities and sponsor reproducibility.

5.4. Model Performance Evaluation

Industry standard classification performance metrics were applied to the trained models. However, this fell short of the initial evaluation goal due to a failure of held-out test set evaluation. This resulted in analysis being performed on validation sets the model has access to during training, as such it is difficult to ascertain the actual performance metrics on each model. It can be reasonably assumed that the models have significant room for improvement, not necessarily limited to training time. Visualisations where possible were produced to highlight claims in the results section.

5.5. Analysis and Implications

Some implications have been explored, though referring to the propagation of more AI models as opposed to the real-world impact within the field (section 4.3). The feasibility of the original goal of a multi-level classification head for each unique EC number has not been reconsidered, due to the sheer amount of training data and class balancing that would be required; it is beyond the scope of my abilities as an undergraduate researcher.

6. Conclusions

6.1 Personal Reflective Statement:

This project was my first introduction to a number of novel computer science and biological concepts. Facilitating research into the underlying mechanisms that make life possible, proteins, has been very interesting. The complexity inherent in such nano-structures is staggering, and the field of bioinformatics goes even beyond this. At the start of this project it was very intimidating as someone who had no experience within this field, however tracing back through the history of Protein Language Models to NLP tasks, and architectures introduced within my data science lectures, I was able to find a common starting point between the project aims and prior knowledge.

If I was to start this project over, I would first take a deeper dive into the Hugging Face documentation, following tutorials to truly understand the internals of the abstracted libraries. The project was frustrating at times, specifically fine-tuning large models in a resource-limited cloud computing environment. Different approaches could have been taken to this problem, starting with researching the mathematics of the size of large language models and how much GPU ram they occupy, instead of the brute-force approach of tweaking a few parameters to reduce GPU usage and restarting the kernel and hoping it'll finish training. Taking a step back to examine each part of my training pipeline and finding the source of the run-time interrupts.

The data preparation portion of the methodology could also have been handled differently. Instead of trying to maintain a chronological order of events, I should have used iterative development and continued to curate the dataset over time. Evaluation of trainer iteration speeds with different sequence lengths could have been attempted, which would have

provided valuable insights into how tokenizer truncation to various lengths could speed up or slow down model training.

In hindsight, I could have experimented more instead of trying to follow a strict path to project completion. Experimentation would also give me a deeper understanding of the technologies used, allowing for novel descriptions in context. In conclusion, *experiment*.

7. References

- [1] Morris, R., Black, K.A. & Stollar, E.J. (2022) ‘Uncovering protein function: from classification to complexes’, *Essays in Biochemistry*, 66(3), pp. 255–285. Available at: <https://doi.org/10.1042/EBC20200108>
- [2] Zhang, H., Lan, J., Wang, H., Lu, R., Zhang, N., He, X., Yang, J. & Chen, L. (2024) ‘AlphaFold2 in biomedical research: facilitating the development of diagnostic strategies for disease’, *Frontiers in Molecular Biosciences*, 11. Available at: <https://doi.org/10.3389/fmolb.2024.1414916>
- [3] Dupree, E.J., Jayathirtha, M., Yorkey, H., Mihasan, M., Petre, B.A. & Darie, C.C. (2020) ‘A Critical Review of Bottom-Up Proteomics: The Good, the Bad, and the Future of This Field’, *Proteomes*, 8(3), p. 14. Available at: <https://doi.org/10.3390/proteomes8030014>
- [4] Li, Z., Yi, Y., Liu, L. & Wu, H. (2024) ‘One step forward for nanopore protein sequencing’, *Clinical and Translational Medicine*, 14(3). Available at: <https://doi.org/10.1002/ctm2.1615>
- [5] The UniProt Consortium. (2024) ‘UniProt: the Universal Protein Knowledgebase in 2025’, *Nucleic Acids Research*, 53(D1), pp. D609–D617. Available at: <https://doi.org/10.1093/nar/gkae1010>
- [6] Alberts B, Johnson A, Lewis J, et al. *Molecular Biology of the Cell*. 4th edition. New York: Garland Science; 2002. *Analyzing Protein Structure and Function*. Available at: <https://www.ncbi.nlm.nih.gov/books/NBK26820/>

-
- [7] Altschul, S.F., Gish, W., Miller, W., Myers, E.W. & Lipman, D.J. (1990) ‘Basic local alignment search tool’, *Journal of Molecular Biology*, 215(3), pp. 403–410. Available at: [10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2)
- [8] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2018) ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.’ Available at: <https://doi.org/10.48550/arXiv.1810.04805>
- [9] Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., Bhowmik, D. & Rost, B. (2022) ‘ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10), pp. 7112–7127. Available at: [10.1109/TPAMI.2021.3095381](https://doi.org/10.1109/TPAMI.2021.3095381)
- [10] Schmidhuber, J. (2014) ‘Deep Learning in Neural Networks: An Overview’
- [11] Schmidt, R.M. (2019) Recurrent Neural Networks (RNNs): A gentle Introduction and Overview.
- [12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. & Polosukhin, I. (2017) Attention Is All You Need.
- [13] Bahdanau, D., Cho, K. & Bengio, Y. (2014) Neural Machine Translation by Jointly Learning to Align and Translate.
- [14] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2018) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
- [15] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., et al. (2019) HuggingFace’s Transformers: State-of-the-art Natural Language Processing. Available at: <https://doi.org/10.48550/arXiv.1910.03771>
- [16] Sri Santhosh Hari. (2025). Understanding Feedback Loops in Machine Learning Systems. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 11(2), 2810–2823. Available at: <https://doi.org/10.32628/cseit25112725>
- [17] McDonald, A.G., Boyce, S. & Tipton, K.F. (2009) ‘ExplorEnz: the primary source of the IUBMB enzyme list’, *Nucleic Acids Research*, 37(Database), pp. D593–D597. Available at: <https://doi.org/10.1093/nar/gkn582>
-

-
- [18] Kaufman, S., Rosset, S. & Perlich, C. (2011) ‘Leakage in data mining’. Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining p.pp. 556–563.
- [19] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. & Chen, W. (2021) LoRA: Low-Rank Adaptation of Large Language Models. Available at: <https://doi.org/10.48550/arXiv.2106.09685>
- [20] Wilson, D.R. & Martinez, T.R. (n.d.) ‘The need for small learning rates on large problems’. IJCNN’01. International Joint Conference on Neural Networks. Proceedings (Cat. No.01CH37222) 1 p.pp. 115–119. Available at: <https://doi.org/10.1109/IJCNN.2001.939002>
- [21] Krogh, A. & Hertz, J. (1991.) ‘A simple weight decay can improve generalization’. Proceedings of the 5th International Conference on Neural Information Processing Systems (NIPS’91). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 950–957. Available at: <https://proceedings.neurips.cc/paper/1991/file/8eefcfd5990e441f0fb6f3fad709e21-Paper.pdf>
- [22] Rainio, O., Teuho, J. & Klén, R. (2024) ‘Evaluation metrics and statistical tests for machine learning’, Scientific Reports, 14(1),. Available at: <https://doi.org/10.1038/s41598-024-56706-x>
- [23] Biderman, D., Portes, J., Ortiz, J.J.G., Paul, M., Greengard, P., Jennings, C., King, D., Havens, S., Chiley, V., Frankle, J., Blakeney, C. & Cunningham, J.P. (2024) LoRA Learns Less and Forgets Less. Available at: <https://doi.org/10.48550/arXiv.2405.09673>

8. Appendices

Appendix A

Implementation of a plotly sunburst chart to visualize hierarchical Enzyme Commission data. Code sourced from the documentation <https://plotly.com/python/sunburst-charts/>, adapted to particular use case. SwissProt dataset was first transformed into a new dataset containing only enzyme commission separated by tier ‘ec_1’, ‘ec_2’, etc

```
EC_df = swissprot_df.copy()
EC_df['ec_numbers_as_list'] = swissprot_df['EC number'].str.split(';') # Create new column "ec_numbers_as_list" that contains a list of EC numbers for each entry with multiple
exploded_df = EC_df.explode('ec_numbers_as_list') # Explode the dataframe so that each entry with multiple EC numbers is split into new rows
exploded_df = exploded_df[['ec_numbers_as_list']] # Select only the column with EC numbers
exploded_df = exploded_df[exploded_df['ec_numbers_as_list'].notna()] # Remove null values
exploded_df['ec_numbers_as_list'] = exploded_df['ec_numbers_as_list'].str.replace('-', '', regex=False) # Remove hypens from EC numbers
# Split the EC numbers into four columns
split_ec_df = exploded_df.copy() # Create a copy of the exploded dataframe
split_ec_df[['ec_1', 'ec_2', 'ec_3', 'ec_4']] = exploded_df['ec_numbers_as_list'].str.split('.', expand=True) # Split the EC numbers into three columns
split_ec_df.head()
```

ec_numbers_as_list	ec_1	ec_2	ec_3	ec_4
1.1.1.1	1	1	1	1

This modified dataset is now used to calculate the number of times each “unique” EC number appears using the first 2 EC numbers. E.g, the EC number 1.1.x.x appears 744 times.

```
# Generate a value column denoting the total number of times each EC number appears in the datasets, focusing on ec_1 and ec_2
value_counts = split_ec_df.groupby(['ec_1', 'ec_2']).size().reset_index(name='value_counts')
# remove whitespace
value_counts['ec_1'] = value_counts['ec_1'].str.strip()
value_counts['ec_2'] = value_counts['ec_2'].str.strip()
print(value_counts)
```

ec_1	ec_2	value_counts
1	1	744
1	10	1

```
import plotly.express as px
import pandas as pd

fig = px.sunburst(
    value_counts,
    path=['ec_1', 'ec_2'],
    values='value_counts',
    color='ec_1'
)

fig.show()
```

Create and display the sunburst chart.

Appendix B

Screenshots of code snippets for the tokenization of large datasets. All code sourced from hugging face documentation:

https://huggingface.co/docs/transformers/en/main_classes/tokenizer

```
[ ] # Load my csv dataset from public huggingface repository
    from datasets import load_dataset

    dataset = load_dataset("JasonTWalker/preprocessed_uniprotkb_binary_label_1024_seq_length", data_files="uniprotkb_binary_label_1024.csv")
```

```
# Get the protbert model and tokenizer from huggingface
from transformers import AutoTokenizer, AutoModel, AutoModelForSequenceClassification

protbert_model = "Rostlab/prot_bert"
tokenizer = AutoTokenizer.from_pretrained(protbert_model)
model = AutoModel.from_pretrained(protbert_model)
```

```
# Get the protbert model and tokenizer from huggingface
from transformers import AutoTokenizer, AutoModel, AutoModelForSequenceClassification

protbert_model = "Rostlab/prot_bert"
tokenizer = AutoTokenizer.from_pretrained(protbert_model)
model = AutoModel.from_pretrained(protbert_model)
```

```
# adjust the tokenizer max length and padding length to 1024 to align with our dataset
print("Tokenizer default model_max_length:", tokenizer.model_max_length ) # will change
tokenizer.model_max_length = 1024
print("Tokenizer adjusted model_max_length:", tokenizer.model_max_length )
```

```
Tokenizer default model_max_length: 1000000000000000019884624838656
Tokenizer adjusted model_max_length: 1024
```

```
# tokenize the dataset and split into train, test, validation with 70/15/15 split
def tokenize_function(examples):
    # Tokenize the sequences using the tokenizer, truncating to a maximum length of 1024 tokens
    tokenized_dataset = tokenizer(examples["Sequence"], truncation=True, padding="max_length", max_length=1024)

    # rename is_enzymatic to label, as this is the expected format for the HuggingFace Trainer
    tokenized_dataset["label"] = examples["is_enzymatic"]

    return tokenized_dataset

tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

Map: 100%  467704/467704 [05:28<00:00, 1659.06 examples/s]

Appendix C

Link to the repository storing the shortened, tokenized, presplit dataset used within model training:

JasonTWalker/tokenized_uniprotkb_binary_label_1024_length_pre_split_random_1_percent

For access to the entire tokenized dataset, preprocessed as outlined within the methodologies section:

https://huggingface.co/datasets/JasonTWalker/tokenized_uniprotkb_1024_sequence_length

Appendix D:

Screenshots from the `trainer.train()` output for each defined model.

Model A:

```

/tmp/python-input-46-3100954136.py:4: FutureWarning: 'tokenizer' is deprecated and will be removed in version 0.10.0
trainer = Trainer(
[515/515 27.42, Epoch 5/5]

Epoch  Training Loss  Validation Loss  Accuracy  Precision  Recall  F1
-----
1      No log          0.674897      0.514286      0.000000      0.000000      0.000000
2      0.671500          0.618283      0.685714      0.692308      0.635294      0.662577
3      0.671500          0.591201      0.697143      0.681818      0.705882      0.693642
4      0.585000          0.543659      0.731429      0.726190      0.717647      0.721893
5      0.585000          0.525423      0.760000      0.736264      0.788235      0.761364

TrainOutput(global_step=515, training_loss=0.6044296412792021, metrics={'train_runtime': 1664.0416,
'train_samples_per_second': 2.458, 'train_steps_per_second': 0.309, 'total_flos': 954713289351680.0,
'train_loss': 0.6044296412792021, 'epoch': 5.8})

```

Model B:

```

/tmp/python-input-17-1901531012.py:4: FutureWarning: 'tokenizer' is deprecated and will be removed in
  trainer = Trainer(
                    [S15/S15 12:18, Epoch 5/5]

Epoch  Training Loss  Validation Loss  Accuracy  Precision  Recall  F1
-----
1      No log          0.688990     0.514286    0.000000    0.000000    0.000000
2      0.685800         0.652453    0.674286    0.694444    0.588235    0.636943
3      0.685800         0.613916    0.691429    0.674157    0.705882    0.689655
4      0.612900         0.578732    0.725714    0.728395    0.694118    0.710843
5      0.612900         0.578952    0.708571    0.702381    0.694118    0.698225

TrainOutput(global_step=515, training_loss=0.6268544821162015, metrics={'train_runtime': 741.395,
train_samples_per_second: 5.517, train_steps_per_second: 0.695, 'total_flos': 9547132893511680.0

```

Model C:

```

[trainer].train()

~/tmp/ipython-input-17-3100954136.py:4: FutureWarning: 'tokenizer' is deprecated and will be removed in a future version.
trainer = Trainer(
  1659/2050 56:10 < 13:15, 0.49 i/s, Epoch 4.0/4.5)

Epoch  Training Loss  Validation Loss  Accuracy  Precision  Recall  F1
1      0.587900      0.594186      0.718973      0.757576      0.606001      0.670017
2      0.486400      0.502861      0.754636      0.780488      0.672673      0.722581
3      0.434600      0.489933      0.813124      0.777473      0.849850      0.812052
4      0.408400      0.479066      0.818830      0.808383      0.810811      0.809595

```

Model D:

trainer = Trainer([[2050/2050 50/21, Epoch 5/5]						
Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.474600	0.500116	0.798859	0.779070	0.804805	0.791728
2	0.409700	0.508056	0.800285	0.760108	0.846847	0.801136
3	0.362500	0.448472	0.840228	0.829851	0.834835	0.832335
4	0.347800	0.432327	0.845934	0.852665	0.816817	0.834356
5	0.328000	0.436924	0.847361	0.840361	0.837838	0.839908

```

TrainOutput(global_step=2050, training_loss=0.4038145837395198, metrics={'train_runtime': 3022.8173,
'train_samples_per_second': 5.414, 'train_steps_per_second': 0.678, 'total_flos': 3.82082288565248e+16,

```
